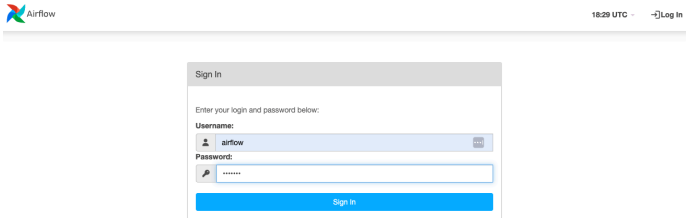
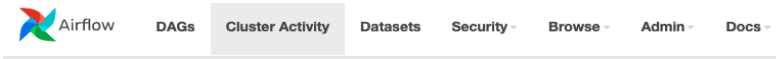
Airflow - Advanced Concepts

Open URL http://108.129.216.179:8080/ (port 8080 of VM IP 108.129.216.179). Login to Airflow UI using default username airflow and password airflow

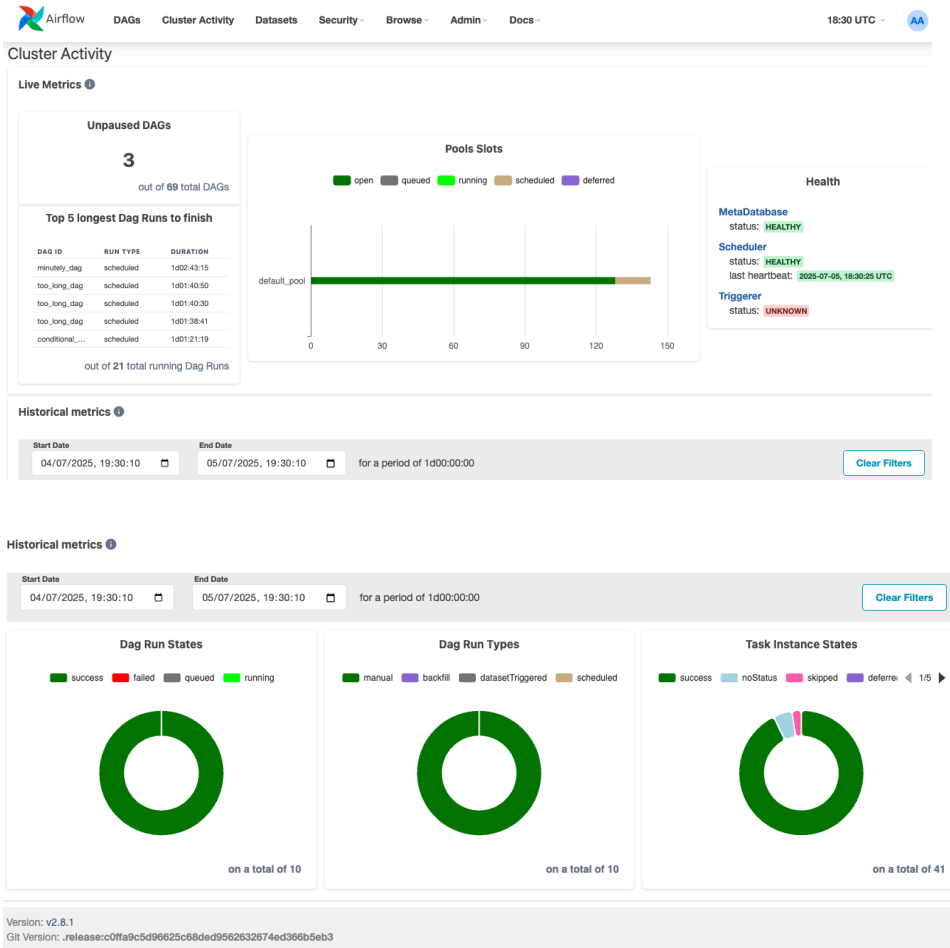


Cluster Monitoring:
Navigate to Admin -> Cluster Activity

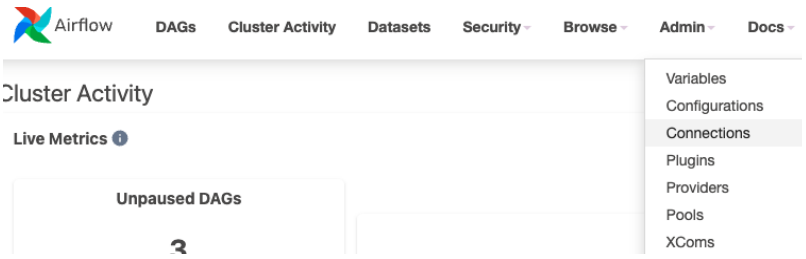


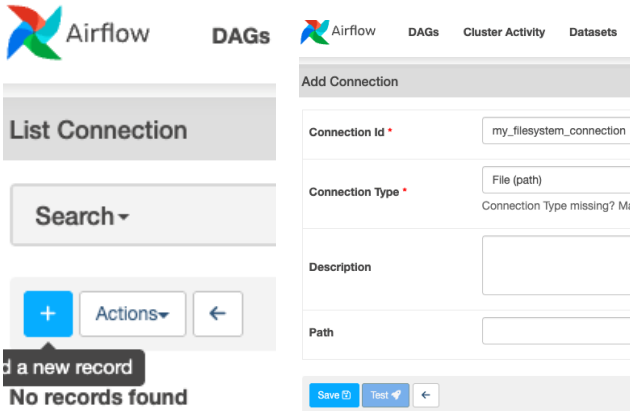
DAGs

Observe Live Metrics and scrolling down the page Historical Metrics etc



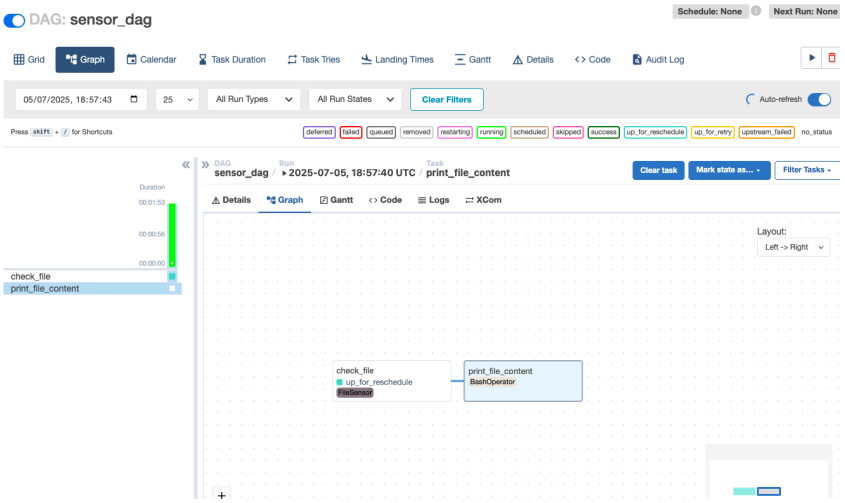
Sensors and Connections:
Navigate to Admin -> Connection and select the + option. Fill in details Conn Id: *my_filesystem_connection* Conn Type: *File (path)* and click *Save* button





Create a sensor_dag.py file in dags/

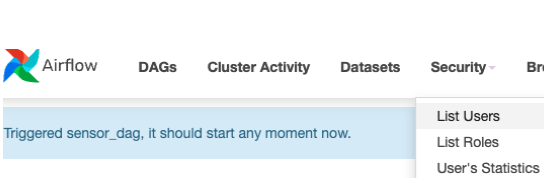
Trigger sensor_dag in Airflow UI. It will go *UP_FOR_RESCHEDULE* while the file is missing



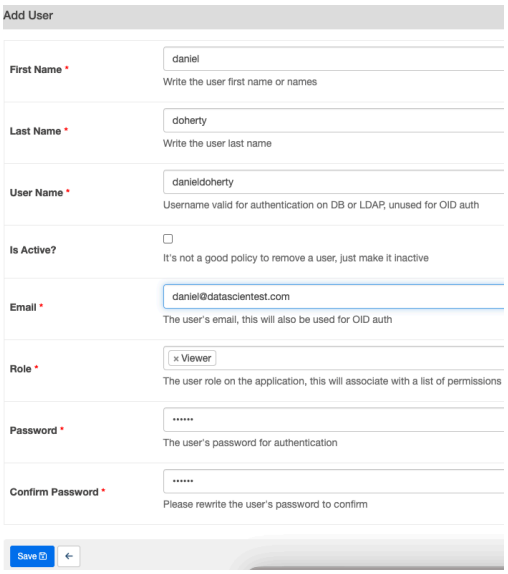
Run the command below. File may be found now and the task should succeed

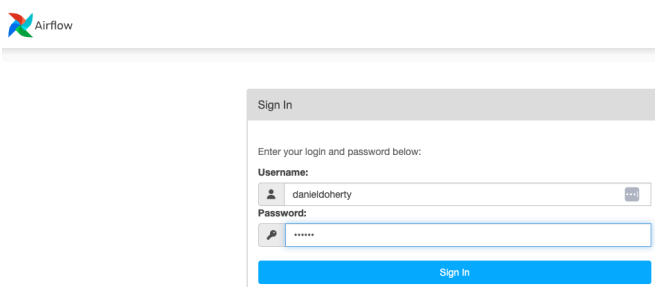
```
ubuntu@ip-172-31-46-112:~/datascientest-data-engineering/airflow/dags$ docker exec -it airflow_airflow-worker_1 bash
default@35a3cfe351c7:/opt/airflow$ echo "hello world" >> /tmp/my_file.txt
default@35a3cfe351c7:/opt/airflow$
```

User Management:
Open Security dropdown and navigate to List Users

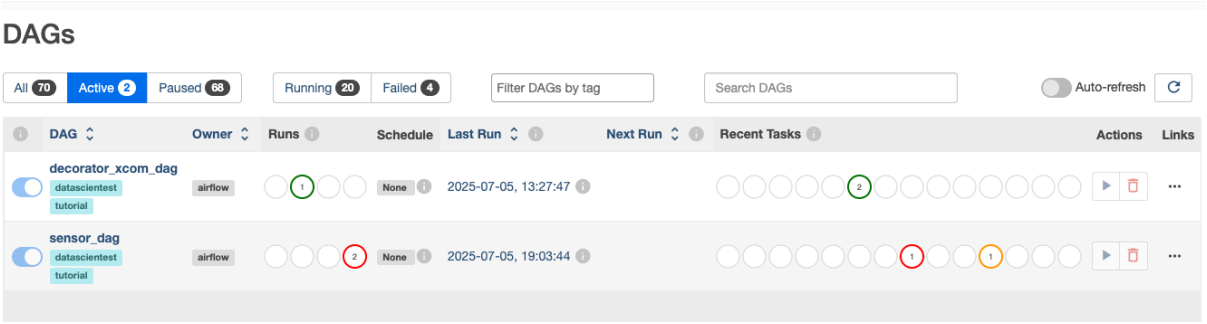


Click + to add a new user and select the Save button. Logout and login with new user details





As user daniel with limited permissions, observe inability to trigger DAGs



Logout and back in again with default username airflow and password airflow again

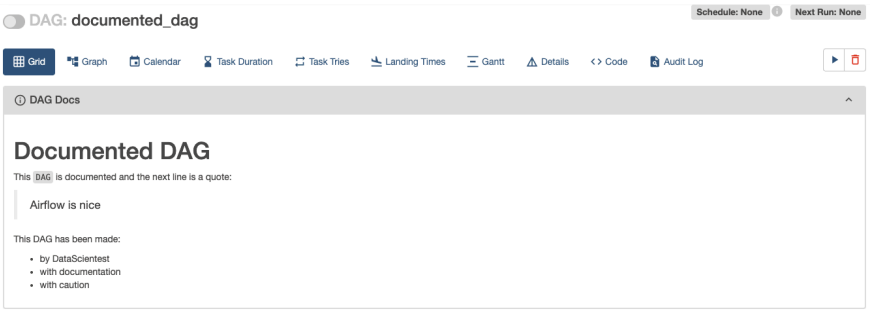
Documentation:
Create a documented_dag.py file in dags/

documented_dag.py U

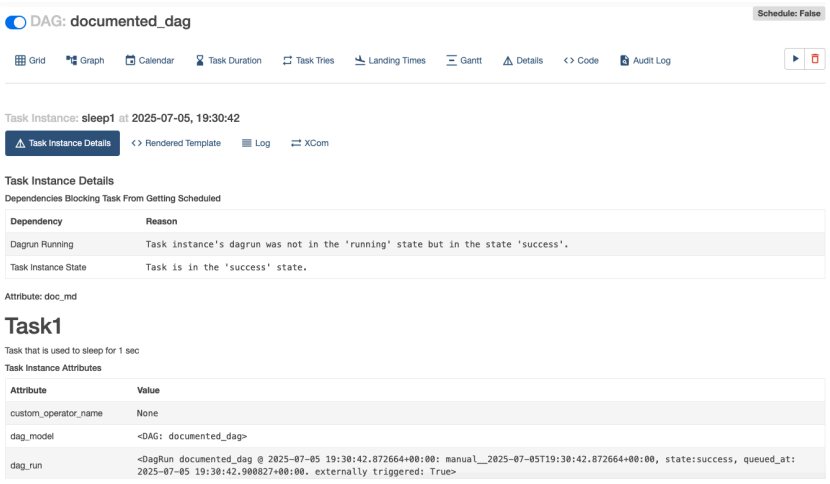
tascientest-data-engineering > airflow > dags > documented_dag.py

```python
from airflow import DAG
from airflow.utils.dates import days_ago
from airflow.operators.python_operator import PythonOperator
import time


def sleep_1_sec():
        time.sleep(1)

with DAG(
    dag_id = "documented_dag",
    doc_md ="""
    # Documented DAG
    This `DAG` is documented and the next line is a quote:

    > Airflow is nice

    This DAG has been made:

    * by DataScientest
    * with documentation
    * with caution
    """,
    start_date = days_ago(0),
    schedule_interval = None
) as dag:

    task1 = PythonOperator(
        task_id = "sleep1",
        python_callable = sleep_1_sec,
        doc_md = """
        # Task1
        Task that is used to sleep for 1 sec
        """,
    )

    task2 = PythonOperator(
        task_id = "sleep2",
        python_callable = sleep_1_sec,
        doc = """
        Task 3
        It has an ugly description.
        """
    )

    task1 >> task2
```

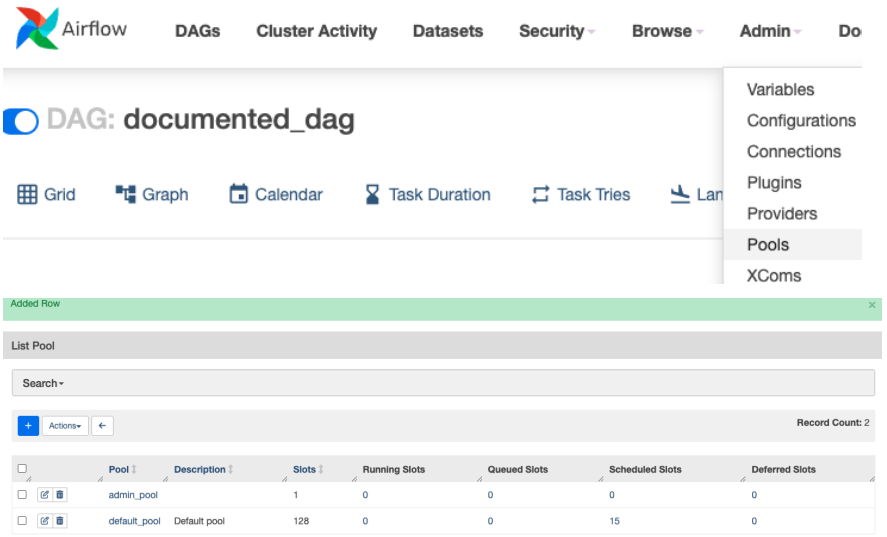
Return to Airflow UI and search for documented_dag. Observe the documentation directly on the DAG documented_dag in DAGs menu



Trigger DAG by clicking the play button. It is also possible to view task documentation via Graph View and selecting a task and then Instance Details



Runtime environment and pools:
Navigate to Admin -> Pools. Create a pool micro_pool with slots 1



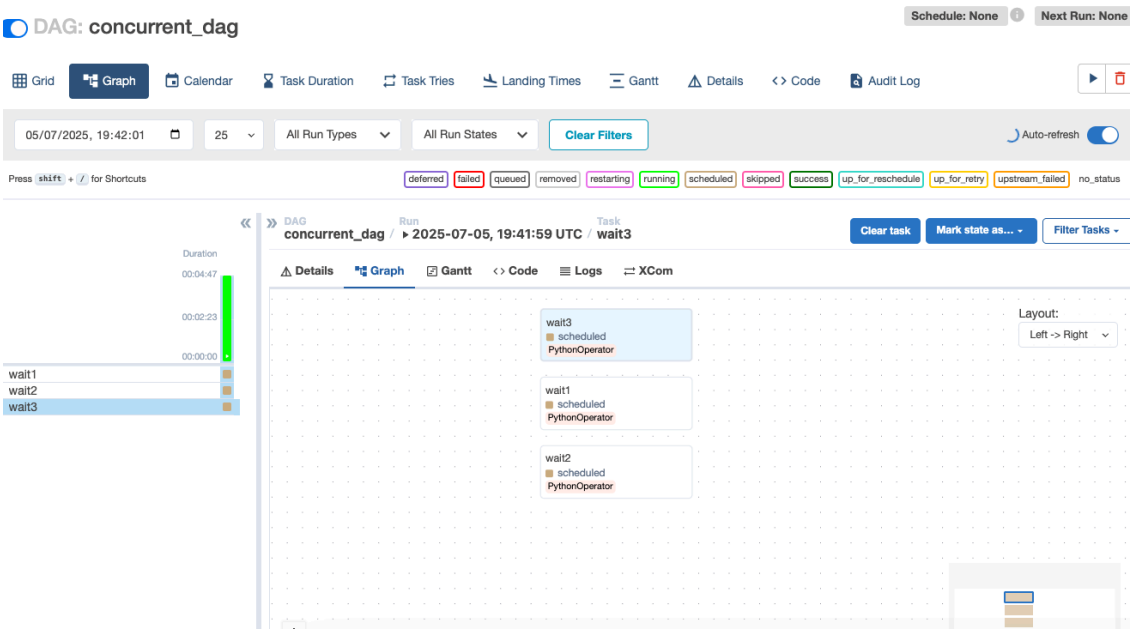
Create a concurrent_dag.py file in dags/

concurrent_dag.py U

atascientest-data-engineering > airflow > dags > concurrent_dag.py

```python
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from airflow.utils.dates import days_ago
import time


def wait_1_minute():
    time.sleep(60)

with DAG(
    dag_id = "concurrent_dag",
    schedule_interval = None,
    start_date = days_ago(0),
    default_args = {
        "pool": "micro_pool"
    }
) as dag:
    task1 = PythonOperator(
        task_id = "wait1",
        python_callable = wait_1_minute,
    )

    task2 = PythonOperator(
        task_id = "wait2",
        python_callable = wait_1_minute,
    )

    task3 = PythonOperator(
        task_id = "wait3",
        python_callable = wait_1_minute,
    )
```

Open Airflow UI, search for concurrent_dag and trigger it. Notice that tasks run one after the other to roughly 3 minutes in total



Comment out the default_args in concurrent_dag.py. Save the file.

concurrent_dag.py U

datascientest-data-engineering > airflow > dags > concurrent_dag.py

```python
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from airflow.utils.dates import days_ago
import time


def wait_1_minute():
    time.sleep(60)

with DAG(
    dag_id = "concurrent_dag",
    schedule_interval = None,
    start_date = days_ago(0),
    # default_args = {
    #     "pool": "micro_pool"
    # }
) as dag:
    task1 = PythonOperator(
        task_id = "wait1",
        python_callable = wait_1_minute,
    )

    task2 = PythonOperator(
        task_id = "wait2",
        python_callable = wait_1_minute,
    )

    task3 = PythonOperator(
        task_id = "wait3",
        python_callable = wait_1_minute,
    )
```

Re-trigger the DAG concurrent_dag. Now all three tasks run in parallel, roughly 1 min in total